

Accuracy Improvement Challenge

Forecasting Weekly Gasoline Production

Major Project 3 of 3 | DSBA — July 2026 Batch

1. Objective & Scope

Improve forecast accuracy for the class-project case study by applying feature engineering and advanced modelling techniques, benchmarked against a documented baseline. Note: the generic brief names this the “Admission Problem,” but the class project actually supplied for this challenge (EDA_Gasoline_Production.ipynb + export.csv) is a weekly gasoline production forecasting case study, not an admissions dataset. This report revisits that real class project, per the task's own instruction to “start from the original class project, same dataset.” The original notebook performed cleaning and exploratory analysis only, with no forecasting model — so this report first establishes a documented baseline before applying the accuracy-improvement techniques the assignment requires.

2. Baseline Model (Task 1)

Train: FY2020–FY2024 (261 weeks). Test: FY2025 (52 complete weeks, the most recent full fiscal year). FY2026 (partial, 44 weeks) is held back entirely as a second, independent validation check in Section 5.

Baseline — Seasonal-Naive (year-over-year): the dataset already contains the actual production for the same fiscal week one year earlier. Using it directly as the forecast is the natural, honest baseline here — it requires no training and already captures the strong weekly seasonality in the data.

Baseline result on FY2025: RMSE = 111,969,591 gallons/week, MAE = 81,223,154, MAPE = 3.11%.

3. Model Enhancement (Task 2)

Feature engineering (leakage-safe)

Every feature below uses only information available before the week being forecast:

Feature	Definition
lag1, lag2	Production 1 and 2 weeks prior
lag52	Production in the same fiscal week one year earlier (the baseline's own signal)
roll4_prior, roll8_prior	Trailing 4- and 8-week rolling averages, ending the week before the forecast week
week_sin, week_cos	Cyclical (sine/cosine) encoding of the 52-week seasonal cycle
trend_idx	Chronological time index, capturing long-run growth or decline
outlier_flag, quarter	COVID-outlier indicator and one-hot fiscal quarter, carried over from the original EDA

Models compared

- Linear Regression, Random Forest and Gradient Boosting (HistGradientBoosting) — trained on the engineered features above.
- Holt-Winters Exponential Smoothing — a classical statistical decomposition (trend + 52-week seasonal), fit only on the historical series; unlike every other model here it never sees the actual prior-year value directly.

- LSTM (deep learning, Extra Work) — a 12-week sequence window augmented with the same seasonal-lag signal (lag52, cyclical week encoding, trend) fed in as auxiliary inputs, for a fair comparison against the ML models.

4. Evaluation (Task 3)

Model	RMSE (gal/wk)	MAE (gal/wk)	MAPE	RMSE Improv.
Gradient Boosting	96,310,339	77,022,000	2.96%	+14.0%
Random Forest	98,426,283	77,795,062	2.99%	+12.1%
LSTM (multi-input)	105,745,478	83,601,454	3.24%	+5.6%
Linear Regression	110,249,848	85,777,337	3.31%	+1.5%
Baseline (Seasonal-Naive)	111,969,591	81,223,154	3.11%	—
Holt-Winters	289,501,611	264,455,170	10.14%	-158.6%

Table 1. FY2025 test-set performance, sorted by RMSE (best first).

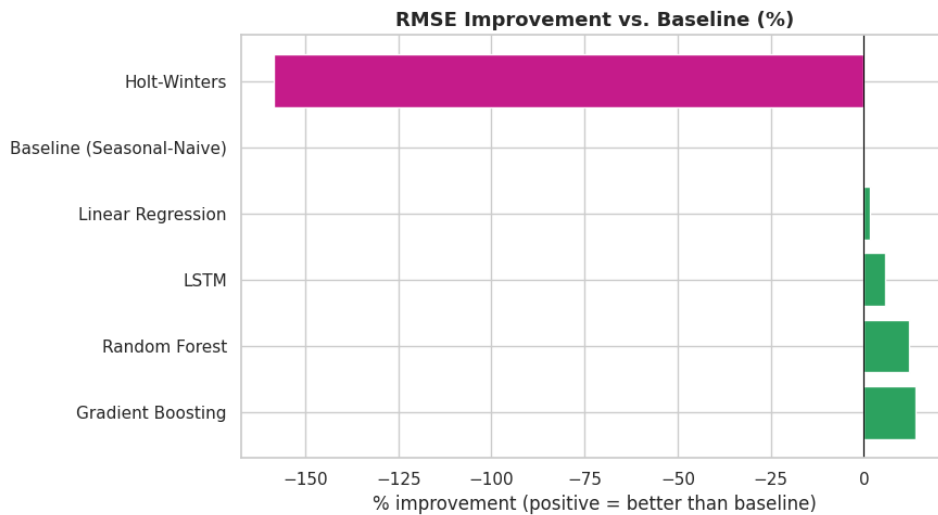


Figure 1. RMSE improvement vs. baseline. Gradient Boosting and Random Forest clearly beat the baseline; Holt-Winters is far worse.

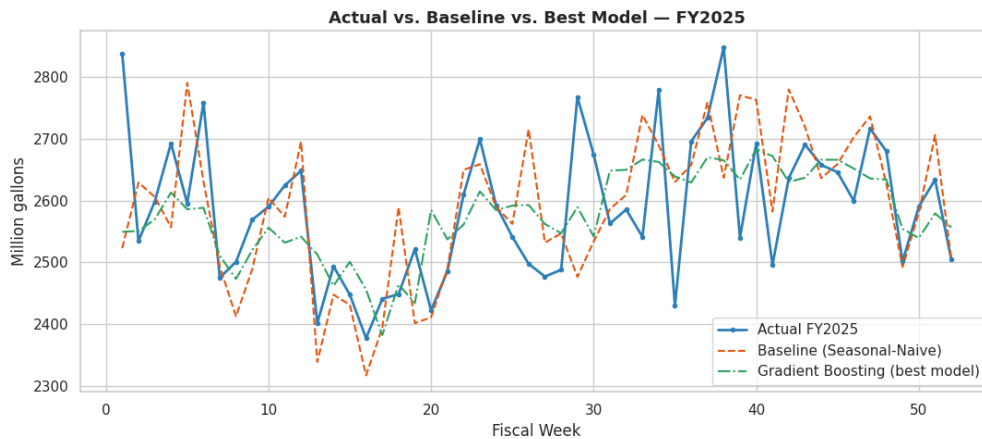


Figure 2. Actual vs. baseline vs. best model (Gradient Boosting) across FY2025 — the enhanced model tracks the smoother underlying pattern with less noise than the baseline's week-to-week swings.

5. Insights (Task 4)

- Gradient Boosting was the best model (+14.0% RMSE, MAPE 2.96% vs. 3.11%), with Random Forest close behind (+12.1%). Both combine the baseline's own year-over-year signal with recent momentum, smooth seasonality and a trend term — learning a small correction to the naive copy rather than replacing it outright.
- Linear Regression improved only marginally (+1.5% RMSE, and MAPE was slightly worse than baseline) — the relationship is not purely linear, so tree ensembles capture more of it.
- The LSTM reached +5.6% once given the same seasonal-lag signal as the ML models — real, but short of the tree ensembles, consistent with deep learning typically needing more than the ~5 years of weekly data available here to reliably out-learn a well-engineered gradient-boosted tree.
- Holt-Winters performed clearly worse than baseline (RMSE ~2.6× higher, MAPE 10.14%). Its forecasts show a systematic under-forecast across the entire test year: its linear trend, estimated from a training window that includes the 2020 COVID production collapse, understates the market's real recovery. The seasonal-naive baseline is immune to this specific failure mode since it never extrapolates a trend — it simply copies a real, recent, comparable data point.
- Retraining Gradient Boosting on FY2020–FY2025 and validating on FY2026 (44 weeks, never seen in any form) still shows a +6.2% RMSE improvement over baseline (92.4M vs. 98.6M gallons/week) — evidence the gain generalizes rather than being an artifact of one favorable test year.

Business implications

- A ~14% cut in weekly forecast RMSE (MAPE 3.11% → 2.96%) directly tightens the buffer production planners need — smaller, more confident safety margins on refinery scheduling, storage allocation and distribution logistics.
- The winning model is a small, fast tree ensemble, not a heavy deep-learning pipeline — cheap to retrain weekly as new actuals arrive, keeping the forecast current with minimal operational overhead.
- The Holt-Winters result is itself an operational insight: any trend-extrapolating forecast method should be re-validated whenever an anomalous shock (like 2020) sits inside its training window, since it can quietly bias the whole forecast horizon in one direction.

Extra Work — Time Series Analysis & LSTM (1-Page Summary)

Time series analysis

Time series analysis studies data ordered in time by separating it into interpretable components — trend (long-run direction), seasonality (repeating patterns, here a 52-week cycle) and residual noise — and using that structure to forecast future values. Classical methods like Holt-Winters explicitly estimate and extrapolate those components. The feature-engineering approach used for the ML models in this project (lags, rolling averages, cyclical week encoding, trend index) achieves a similar goal implicitly, by giving a general-purpose regression model enough historical context to learn the patterns itself, rather than imposing a fixed decomposition structure.

LSTM (Long Short-Term Memory)

An LSTM is a recurrent neural network designed to learn patterns in sequences by maintaining an internal memory state that carries information forward across many time steps, using learned gates to control what to keep, forget and output at each step. This makes it well suited, in principle, to time series with long-range dependencies.

In this project, the LSTM was given a 12-week rolling window of production plus the same seasonal-lag features as the ML models (rather than the raw series alone, which underperformed even the baseline in early testing), and reached a +5.6% RMSE improvement — real, but smaller than the tree ensembles. This illustrates a common real-world pattern: deep learning typically needs substantially more historical data than the ~5 years available here to reliably out-perform well-engineered tree-based models on tabular/sequential forecasting problems of this size. LSTMs tend to earn their advantage on longer, higher-frequency series (e.g., daily/hourly data spanning many years) or where relationships are too complex for hand-engineered lag features to capture.

Practical takeaway: reach for classical decomposition (Holt-Winters/ARIMA) or engineered-feature ML models as a strong, fast first line for weekly business series like this one; reserve LSTM/deep learning for cases with materially more historical data or where simpler methods demonstrably fall short.